5. Construct a scheduling program with C that selects the waiting process with the highest priority to execute next.

PROGRAM:

```
#include <stdio.h>

struct priority_scheduling
{
    char process_name;
    int burst_time;
    int waiting_time;
    int turn_around_time;
    int priority;
};

int main()
{
    int number_of_process, total = 0, position;
    float average_waiting_time, average_turnaround_time;
    int ASCII_number = 65;

    printf("Enter the total number of Processes: ");
    scanf("%d", &number_of_process);

    struct priority_scheduling process[number_of_process], temp_process;

    printf("\nEnter Burst Time and Priority:\n");
```

```
// Input
for (int i = 0; i < number_of_process; i++)
{
    process[i].process_name = (char)ASCII_number++;

    printf("\nProcess %c\n", process[i].process_name);

    printf("Burst Time: ");
    scanf("%d", &process[i].burst_time);

    printf("Priority: ");
    scanf("%d", &process[i].priority);
}

// Sorting based on priority (Higher value = higher priority)
for (int i = 0; i < number_of_process; i++)
{
    position = i;
    for (int j = i + 1; j < number_of_process; j++)
    {
        if (process[j].priority > process[position].priority)
            position = j;
    }

    temp_process = process[i];
    process[i] = process[position];
    process[position] = temp_process;
```

```
}

// Waiting Time
process[0].waiting_time = 0;
for (int i = 1; i < number_of_process; i++)
{
    process[i].waiting_time = 0;
    for (int j = 0; j < i; j++)
    {
        process[i].waiting_time += process[j].burst_time;
    }
    total += process[i].waiting_time;
}

average_waiting_time = (float)total / number_of_process;

// Turnaround Time
total = 0;
printf("\nProcess\tBT\tWT\tTAT\n");

for (int i = 0; i < number_of_process; i++)
{
    process[i].turn_around_time =
        process[i].burst_time + process[i].waiting_time;

    total += process[i].turn_around_time;  // ✅ FIXED
```

```
        printf("%c\t%d\t%d\t%d\n",
                process[i].process_name,
                process[i].burst_time,
                process[i].waiting_time,
                process[i].turn_around_time);
    }

    average_turnaround_time = (float)total / number_of_process;

    printf("\nAverage Waiting Time = %.2f", average_waiting_time);
    printf("\nAverage Turnaround Time = %.2f\n", average_turnaround_time);

    return 0;
}}
```

OUTPUT:

```
Output                                                    Clear
Enter number of processes: Enter number of processes: 4


Enter Burst Time and Priority for Process P1: 5 2

Enter Burst Time and Priority for Process P2: 3 1

Enter Burst Time and Priority for Process P3: 8 4

Enter Burst Time and Priority for Process P4: 6 3

Process Burst Time  Priority    Waiting Time    Turnaround Time


=== Code Execution Successful ===
```